AgentWorkShop: A Modular Task-to-Device Integration Framework for Industrial Agents

***Abstract*—Deploying industrial agents across production scenes requires scene-specific signal and protocol mappings while keeping task context, device dispatch, and resulting evidence aligned. This paper presents AgentWorkShop, a modular task-to-device integration framework that composes task adapters, production context, named operations, semantic input/output, runtime checks, protocol drivers, and service-owned records. Registered task routes pass through the collaboration runtime; direct operation calls enter the shared dispatcher without creating a task record. The design keeps approval, command readback, and subsequent process observations distinct while linking them to the active production context. We evaluated registered task paths and direct operation calls in cast-film and annealing simulators with different signal and protocol mappings. Six sequential model-mediated task executions and two fixture-backed command-line tasks preserved their route-specific record links; direct calls retained operation, readback, and run-tagged sample records. Every current-route sample remained below its target band. A mock-state approval-context test refused 12 stale changes before writes, whereas an ablation allowed six product/run changes through. A separate v0.8.3 replay of one documented coating-line blueprint passed 23 functional verification checks and confirmed one simulator write. Together, these results demonstrate integration behavior for selected preconfigured paths, one simulated onboarding configuration, and a software dispatch guard. They do not establish process improvement, configuration reuse, general portability, or comparative platform benefit.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, semantic data integration.**

I. Introduction

INDUSTRIAL-AGENT deployments must connect each task to its production context, process signals, and equipment interfaces. Existing work addresses end-to-end automation control, operator-reviewed control-sequence generation, modular factory skills, and production scheduling [1]–[4]. Other studies target PLC code generation, context-aware control, resource capability discovery, shop-floor coordination, digital-twin fault handling, natural-language machine access, and robotic-line control [5]–[10]. Across production lines, agent roles, signal maps, protocols, objectives, and product/run context vary. An integration architecture must accommodate these differences around shared task and operation services while keeping each action tied to its active production context. Recent industrial-agent architectures add AAS-grounded OPC UA actions with signed approval, P&ID-grounded proposal checks, and governed sensor–MES–ERP evidence streams [11]–[13]. These works address asset-specific execution, process validation, or closed-loop analytics. At their software-integration boundary, this paper examines a specific implementation requirement: task routes and direct tool callers should invoke one shared named operation handler while retaining evidence appropriate to each route. Task routes can link task, approval, command receipt, and process observation; a direct call has operation records but no task edge. The contribution concerns the implementation and evaluation of this contract; equivalent behavior may also exist in prior platforms.

Consider two production scenes with different process signals and interfaces: cast-film thickness control and annealing-zone temperature control. An agent task in either scene needs production context, typed signal access, and a route from a named operation to the configured device protocol. The framework must also distinguish a command receipt from a subsequent process observation. The shared task and operation services can therefore be configured with scene-specific signal maps, protocols, and limits while retaining a common execution path. The evaluation examines this path in two simulator scenes and replays a documented coating-line onboarding workflow.

This paper presents AgentWorkShop as a modular framework for task-to-device integration in industrial agents. It connects collaborative tasks through a registered harness interface to production context, manufacturing execution system (MES) data, analysis and modeling services, data-acquisition (DAQ) and write-control (DCW) nodes, and protocol-specific drivers. Its interfaces include command-line, web, software development kit (SDK), Representational State Transfer (REST), Model Context Protocol (MCP), and Agent-to-Agent (A2A) surfaces. Task and chat routes enter the collaboration runtime, whereas a direct REST tool-invocation route can call the shared operation dispatcher without creating a task. Scene blueprints define signal maps, objectives, and guards; channel profiles, tool policies, and agent bindings scope operations. Agents and callers invoke named operations through shared runtime checks; the framework does not choose routes autonomously and retains proposals, command confirmation, and process observations as distinct records. Fig. 1 presents the system architecture, and Fig. 2 shows the cross-module task flow. Section IV follows task context through shared dispatch, device admission, approval, and route-specific evidence.

The evaluation asks whether the registered-harness task route, a fixture-backed command-line task route, and direct REST calls reach the same named operation across two configured simulator mappings while preserving the records applicable to each route. A fixed-call replay separates task-path behavior from an external model decision, and a mock-state experiment examines the approval-bound context check before software dispatch. A separate v0.8.3 replay onboards a documented coating-line blueprint with Modbus TCP and MES REST interfaces in a fresh isolated instance. These experiments cover selected route contracts and one configuration-driven onboarding workflow; they do not establish arbitrary adapter interchangeability or process improvement. Section V

reports the protocols, results, and evidence limits.

The paper makes two contributions:

1) It implements a modular execution path that brings production context, semantic I/O, operation checks, configured protocol drivers, and service-owned records to one named-operation boundary. Task routes retain linked task, approval, receipt, and observation records where available; direct calls retain operation records without creating a task. For approved recipe actions, the system binds approval to the active line–product–recipe–run tuple and checks it again after asynchronous authorization, just before software dispatch. This software check is not an atomic lock across device I/O, a new protocol, a safety-verification method, or a process-control algorithm.
2) It reports route-specific record continuity for registered-harness and command-line task routes and direct REST calls across cast-film and annealing simulator mappings, a documented coating-line onboarding replay, and a mock-state test and ablation of the context check. The onboarding replay checks protocol access, heterogeneous MES data, channel configuration, acquisition, and an approved simulator write; process readings are reported separately from these integration outcomes.

Section II reviews related industrial-agent and integration work. Section III presents the design rationale, architecture, and cross-module execution flow; Section IV explains module responsibilities and their integrated operation. Section V describes the experiments and results, and Sections VI and VII discuss the evidence limits and conclude the paper.

II. Related Work

This section relates AgentWorkShop to plant-facing industrial agents and to production abstractions for context, tool access, and operational constraints. Table I compares two aspects of the integration contract: whether task and direct-operation entries share a named handler, and how operational evidence is associated. Entries describe features reported in the cited sources. "Not reported" does not imply absence; evaluation descriptions retain each source's metrics and success criteria.

A. Industrial Agents and Plant Integration

Industrial-agent systems, including large language model (LLM)-based systems, span end-to-end automation control, supervisory services, programmable logic controller (PLC) interfaces, manufacturing coordination, and operator-facing machine access. Xia et al. present an LLM agent framework for end-to-end industrial automation control, using structured prompts and event-driven context for production planning and operation [1]. Huang et al. present a multimodal architecture whose stated focus is automated control generation for industrial cyber-physical systems [2]. Biskupovic et al. study agent-based supervision for service-oriented industrial systems [14]; Agents4PLC and InstructMPC address PLC code generation and human–LLM context-aware control, respectively [5], [6]. Manufacturing studies consider dynamic resource-capability exploration [7], multi-agent shop-floor coordination [8], and digital-twin-supported fault handling [9]. Natural-language machine access [10] and real-time LLM control of a robot-operated serial line [4] extend this work to operator interfaces and robotic-line control. Together, these studies address different operational contexts, interfaces, and allocations of responsibility between agents, operators, and controllers.

The closest architecture-level comparison is Xia et al., who connect cognitive, digital-twin, and physical-automation layers and organize twin information as state snapshots, event messages, and plan sequences [15]. Other nearby designs expose factory-module operations through MCP with state-based tool filtering, constrain manufacturing plans, or compose semantic recipes from OPC UA skills [3], [16], [17]. Several recent systems also combine agent execution with approval, industrial data, or audit records. Graul et al. ground proposals in the Asset Administration Shell (AAS) and route approved actions through OPC UA Methods, with signed approvals and audit logging [11]. Schall separates advisory language-model agents from process and instrumentation diagram (P&ID)-grounded verification and execution, and evaluates structural checks across five simulated process scenarios [12]; its context includes supervisory control and data acquisition (SCADA) and standard operating procedure (SOP) information. Lin and Chen build a governed sensor–MES–ERP digital thread for closed-loop monitoring and control on real manufacturing records [13]. These works address asset-specific approval, process validation, or closed-loop analytics. The complementary question here is whether a modular task-to-device implementation can route registered-harness and command-line task paths and a direct caller through a shared named handler while preserving the records applicable to each route; its approved recipe path also rechecks the line–product–recipe–run tuple before software dispatch. The route study examines two preconfigured simulator mappings, and a separate blueprint replay checks one documented coating-line onboarding workflow through the existing interfaces. Table I positions these mechanisms by reported features rather than comparative performance.

The operation checks complement safety-aware decision support [18] but do not replace independent process protection.

B. Production Context, Agent Interfaces, and Automation

Industrial integration also depends on abstractions that are not specific to agent architectures. The Model Context Protocol (MCP) specifies interoperable tool discovery and call mechanisms [19]; the operation-level scope, policy, grant, and binding checks discussed here are AgentWorkShop's own dispatcher rules, distinct from MCP transport authorization. In parallel, ISA-88 and ISA-95 describe recipes and production operations [20], [21]. Run-to-run methods connect process observations to successive production runs [22]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [23], [24]. Together, these works provide complementary abstractions for production context, tool interaction, and information flow. AgentWorkShop composes such abstractions with its task runtime, operation checks, semantic I/O, and route-specific records; this paper evaluates selected paths through that composition rather than proposing a replacement for the underlying standards.

TABLE I
REPORTED SHARED-ENTRY HANDLING AND EVIDENCE STRUCTURE IN SELECTED PLANT-FACING STUDIES. SHARED HANDLING MEANS THAT TASK AND DIRECT-OPERATION ENTRIES RESOLVE TO THE SAME NAMED HANDLER; "NOT REPORTED" IS NOT A CLAIM THAT A SYSTEM LACKS THE FEATURE. EVALUATIONS RETAIN SOURCE-SPECIFIC SCOPE AND CRITERIA.

System	Shared handler for task and direct entries	Reported operation and evidence structure	Reported evaluation
Xia et al. [15]	Not reported	Cognitive, twin, and physical layers; twin snapshots, event messages, and plan sequences	Robotic automation and process-simulation cases
Köhle et al. [3]	Not reported	Injected live state and state-based filtering; MCP-to-OPC UA operations and MQTT coordination	Nine simulated production challenges
Vieira da Silva et al. [16]	Not reported	Constrained multi-step plans over MCP manufacturing functions	Laboratory multi-step process
Köhle et al. [17]	Not reported	Semantic recipes composed from OPC UA skills; virtual commissioning	Unity assembly-workstation proof of concept
Graul et al. [11]	Not reported	AAS-grounded proposals, signed human approval, PLC-validated OPC UA Methods, and immutable audit log	Two laboratory PoCs; single scripted runs
Schall [12]	Not reported	P&ID-, SCADA-, and SOP-grounded proposal verification with gated execution	Five simulated scenarios; 43 proposals and 50 LLM runs
Lin and Chen [13]	Not reported	Governed sensor–MES–ERP digital thread with SOP-, risk-, and capability-aware action gating	25,275 real records; operational metrics
This work	Implemented; selected routes tested	Shared named dispatcher; task, approval, receipt, and sample links on task routes; receipts, readbacks, and run-tagged samples for direct calls	Two simulator mappings; one coating-line blueprint onboarding replay; context-guard test and ablation

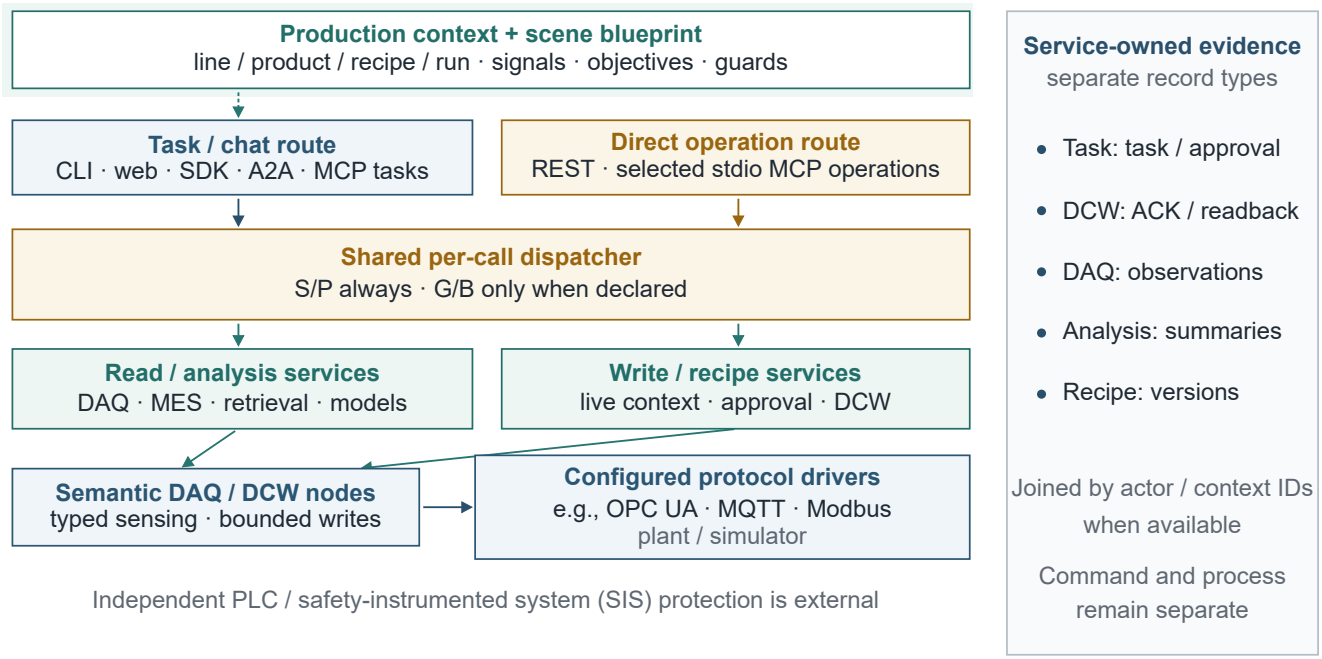


Fig. 1. Architecture and record ownership. Task/chat routes enter the collaboration runtime; direct REST and stdio-MCP calls enter shared dispatch checks without creating a task. Dashed arrows denote configuration. Write routes add approval and four-layer admission; the shared dispatcher always checks scope and channel policy, with grant and resource-binding checks applied when required by the operation. Owning services retain distinct records. Only REST was evaluated; stdio-MCP is shown but untested. Independent PLC and safety-instrumented protection is external.

III. SYSTEM DESIGN

AgentWorkShop is designed for industrial tasks whose agents, production contexts, interfaces, and process signals vary between deployments. Its architecture organizes these variations around shared task and operation services, checks each proposed operation against the active production context

before device routing, and retains route-specific evidence that distinguishes a command receipt from a measured process response.

A. Design Rationale and Requirements

Five requirements—scene variability, harness variability, operation authority, outcome observability, and extensibility—organize six design rules (D1–D6). These are deployment constraints drawn from the target workflows. D1 stores scene settings as versioned data; D2 provides registered harness, tool, and driver interfaces; D3 separates proposal from execution; D4 treats the tool catalog as discovery rather than permission; D5 routes actuator writes through the common DCW admission point; and D6 retains evidence by operation.

The requirements trace to implementation as follows. Scene variability maps to D1, scene blueprints, and context resolution; harness variability and extensibility map to D2, the registered harness and driver interfaces; operation authority maps to D3–D5, the shared dispatcher, grants, bindings, and write admission; outcome observability maps to D6, route-specific receipts, and observations. These interfaces are design extension points; Section V tests selected task and direct-operation routes and one documented blueprint-onboarding workflow. AgentWorkShop supplies task and operation services; agent strategy and independent process protection remain outside this design.

B. Layered Architecture

Fig. 1 summarizes the access routes, shared dispatch path, production services, semantic I/O, configured drivers, and service ownership of records. Task and chat routes enter the collaboration band, which contains workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. A *task route* is an execution path through this runtime; a *harness adapter* connects an agent engine to its task execution interface. Direct REST tool calls and the stdio MCP operation bridge bypass task execution in that band (the gold lane in Fig. 1) and enter the shared governance block without creating a task or starting a harness turn; MCP management and task methods remain on their service routes. Direct calls resolve current route state at invocation, without an agent-turn prompt or a task record. The governance block projects the operation catalog and applies the per-call checks; write routes add the approval card and the admission interval. The context block supplies the active line–product–recipe–run tuple to the gate and scene maps, objectives, and guards to the handlers; its links are configuration, not execution, and the runtime assembles prompt-time context from the same records. The service band provides acquisition, write control, analysis, retrieval, and optional model services behind a route-handler interface that selects a service adapter. The field-I/O band resolves semantic nodes to built-in or plugin drivers for a plant or simulator.

Each band has a distinct responsibility. Task/chat routes enter the task runtime; direct operation routes enter the dispatcher; and both use shared operation checks where applicable. A harness adapter supplies agent-specific interaction on task paths, semantic nodes hide protocol details from operations, and the driver registry provides the device-facing extension point. Owning services retain task, command, observation, and version records separately and link them to available actor and production context. No single status conflates these outcomes.

C. Cross-Module Execution Flow

Fig. 2 summarizes a task turn from entry through operation execution, with route-specific evidence retained by owning services. Registered harness adapters receive a scoped tool catalog during setup and supported registry refreshes; the runtime assembles prompt-time production context for each agent turn. This context is grounding information, not a task-pinned transaction. For each returned tool call, the shared dispatcher rechecks invocation scope and channel policy and, for industrial operations, current grant and binding conditions before handler selection. The selected handler then applies route-specific guards, active-run checks, approval revalidation, and device-I/O steps as applicable. Command receipts and later process observations remain distinct records.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a, c, u) = \bigwedge_{X \in \mathcal{C}(u)} X(a, c, u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, runtime grant, and the agent's binding to referenced resources, respectively. $\mathcal{C}(u) \subseteq \{S, P, G, B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is omitted only for an explicitly grant-exempt industrial tool, and B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured. This condition is necessary for handler selection, not sufficient for actuation: the selected route applies its own guards and approval, then rechecks authority, context, and applicable limits before dispatch.

For direct REST invocation, the API establishes a and c before applying these checks. An agent token must match the requested agent identifier; otherwise, an authenticated user must be the channel owner or an administrator. The manager loads the channel-agent record to obtain the agent identity, role, and channel, then rechecks invocation scope and channel tool policy. Industrial operations continue through the declared shared grant and binding checks. Handlers resolve live production state from request identifiers and owning services, supplying the authorization and production context for direct dispatch.

Fig. 3 gives reference pseudocode for the registered-harness task path, not one literal orchestrator function. Direct REST tool invocation bypasses the task runtime and harness but enters the same shared dispatcher. Route-specific timing, approval, and evidence rules are detailed in Section IV.

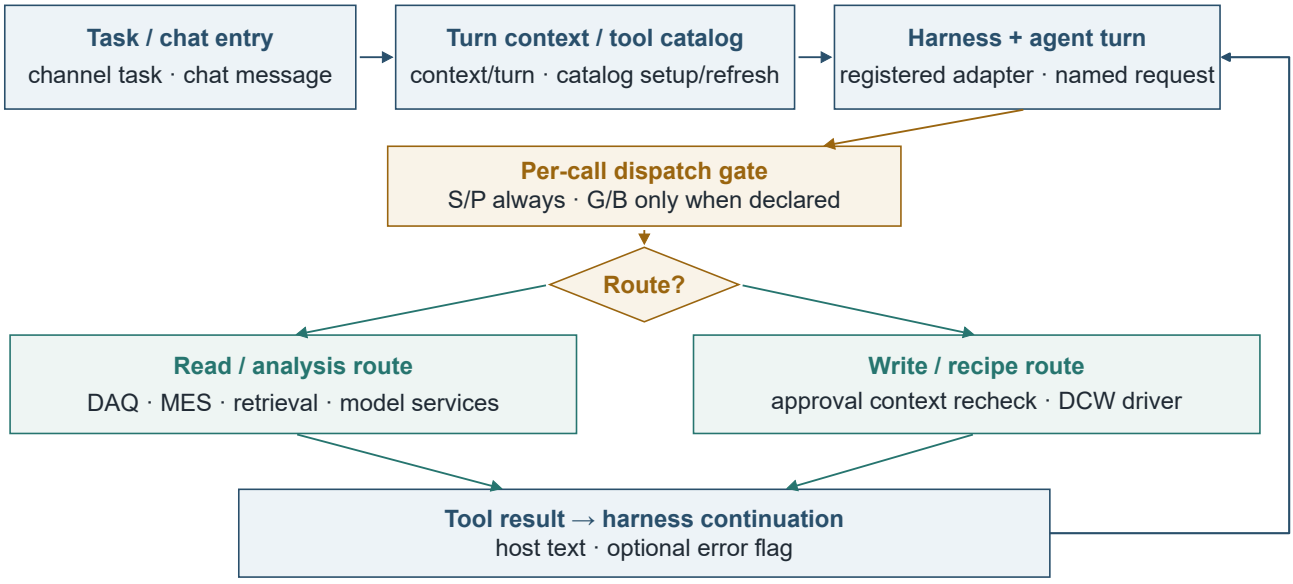


Fig. 2. Registered-harness task flow. The runtime invokes the adapter with a scoped catalog. Scope and channel policy are checked for every call; grant applies to non-exempt industrial operations, and binding applies when resource references are declared. Failed checks return tool errors. The read path retrieves data. The write path applies route guards, approval and context revalidation, DCW admission, and receipt grading; a later process observation is separate. Results return to the harness. Checks occur at the software boundary and reserve no equipment state.

Require: task q, channel c, bound agent a, registered scoped catalog T
Ensure: task result; any retained evidence remains with its owning service
$x_t \leftarrow \textsc{ResolvePromptContext}(c, q)$
$m \leftarrow \textsc{RetrieveMemory}(q)$
$p \leftarrow \textsc{AssemblePrompt}(q, x_t, m)$
$h \leftarrow \textsc{InvokeRegisteredHarness}(a, p, T)$
for all tool request u returned by h **do**
  $g \leftarrow \textsc{CheckScopeAndChannelPolicy}(a, c, u)$
  if not g **then**
    $\textsc{EmitToolErrorToHarness}$(h,u); **continue**
  end if
  if u is an industrial operation **then**
    $v \leftarrow \textsc{CheckApplicableGrantBinding}(a, u)$
    if not v **then**
      $\textsc{EmitToolErrorToHarness}$(h,u)
      continue
    end if
  end if
  $k \leftarrow \textsc{SelectHandler}(u)$
  $r \leftarrow \textsc{ExecuteRouteHandler}(k, a, c, u)$ ▷ handler resolves live state and may persist route-specific records
  $\textsc{EmitDeclaredToolResultToHarness}$(h,r)
end for
return task state and tool results

Fig. 3. Reference pseudocode for the registered-harness task path. Direct operation invocation is not shown. Evidence persistence is route-specific and remains with the owning service; the shared tool result does not carry a generic evidence envelope.

IV. Module Functions and Integration

The design in Section III is realized through three cooperating functions: task and context assembly, shared operation dispatch, and controlled device access with route-specific evidence. We follow one agent action through them.

A. Task, Harness, and Context

Task and chat entry points create or join a workspace channel. The task runtime manages membership, task state, and shared execution context; a harness registry adapts each supported agent engine to a common execution interface. At each agent turn, the prompt builder assembles the task, active scene description, production context, channel profile, agent binding, and retrieved memory. This keeps engine-specific interaction behind the harness interface while preserving the current line and operation context for the agent.

Direct REST and selected stdio-MCP operation calls reach the same dispatcher without a task or harness turn. REST authentication and identity resolution follow Section III-C; each handler reads live route state at invocation. For either entry path, prompt context and tool-catalog visibility describe available information and capabilities; neither grants write authority.

B. Shared Dispatch and Device Admission

The dispatcher is the shared operation boundary. It checks invocation scope and current channel policy, then verifies the agent binding, runtime grant, and ownership of referenced resources before selecting a handler. Handlers revalidate route-specific state at execution time; a previously displayed tool catalog does not preserve authority after a policy or grant

changes. Scene blueprints define device and signal maps, objectives, and process guards. Capability profiles and tool policies narrow available operations, while bindings constrain accessible nodes, controls, and recipes. Optional DAQ, MES, analysis, retrieval, optimization, and digital-twin services are invoked through registered handlers; the framework does not prescribe a single optimization strategy.

Actuator writes converge on the shared write-control (DCW) admission point. For an online node write, the admissible value interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{2}$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{B}_n$, $\mathcal{P}_n$, and $\mathcal{R}_n$ are the baseline, active-product, and active-recipe limits. Missing optional limits impose no restriction; an empty intersection rejects the request and identifies the excluding layer. Recipe dispatch validates the replacement recipe against the hard, baseline, and product limits rather than the previous recipe window. Step, interval, and hold-window checks also constrain each route. These software checks bound commands but do not replace independent programmable-logic-controller or safety-instrumented protection.

C. Approval, Execution, and Evidence

Routes add timing and approval checks appropriate to their operation. Online node writes enforce a 60 s minimum spacing; recipe trials use a per-line cadence (300 s by default), and recipe intervals are anchored to approval. Manual approval binds an actor and proposed operation to candidate values, applicable limits, expiry, and the active line–product–recipe–run tuple. After approval, the handler rechecks authority and compares the current tuple with the approved tuple before entering DCW dispatch. The default approval wait expires after 180 s. A changed recipe definition requires a new `recipe_apply` proposal. These checks reject stale requests at the software dispatch boundary; they do not lock equipment state across network or device I/O.

Fig. 4(a) illustrates the four-layer value interval in Eq. (2) using an archived RPM configuration. Panel (b) shows a symbolic approval followed by authority and context rechecks; its values are illustrative, not experimental measurements.

A `recipe_trial` illustrates the operation sequence: the handler checks the bound recipe, active run, candidate parameters, and route timing; if required, it requests approval; after revalidation, it submits a bounded batch through the configured driver and returns a per-parameter receipt. The caller obtains later DAQ observations separately and may then update, apply, or roll back a recipe. A trial neither saves nor applies a recipe version. The driver registry maps semantic controls to configured protocol drivers, keeping protocol details out of agent operation arguments.

Receipts and observations answer different questions. A command result is graded as readback-verified, transport-acknowledged, or unverified; a readback mismatch can trigger up to three reads at 2 s intervals. Recipe batches retain per-parameter verification outcomes. DAQ stores subsequent timestamped process observations, while recipe versions, approvals, and task records remain service-owned records linked where a route supplies the corresponding identifiers. A verified setpoint confirms the command path, not target attainment. This distinction defines the evidence chain evaluated in Section V.

V. Experimental Evaluation

The experiments examine four aspects of AgentWorkShop: route-specific record continuity across two configured simulator scenes, completion of one documented blueprint-onboarding workflow, rejection of stale approval context before software dispatch, and the process samples associated with the tested routes. These outcomes are kept separate. All plant-facing checks used software simulators; none involved a physical plant or independent process replication.

A. Test Design and Protocol

For a registered task, the integration criterion is a task linked to its approval, readback-verified command receipt, and a process sample tagged to the active run. A direct operation call has no task edge; its criterion is a verified receipt/readback and an active-run-tagged sample. The fixed-call harness replay checks record links, not task completion. The context guard is assessed by whether a changed line, product, recipe, or run is refused before a driver write.

The route-continuity study used the v0.8.0 application with cast-film and continuous-annealing simulator scenes. Cast film wrote screw speed through OPC UA and observed thickness through MQTT (target: 48–52 μm). Annealing wrote the zone-3 setpoint through Modbus TCP and observed strip temperature (target: 700–760 °C). The registered-harness path ran three sequential tasks per scene on one persistent simulator; each task used its own production and task context, while process state carried forward. After each verified receipt, the runner waited 20 s, probed and ingested one process sample, queried it by active-run ID, and then checked task state. Thus, task-completion-to-sample latency is unavailable. The fixed-call replay, direct REST call, and fixture-backed command-line task ran once per scene in separate isolated environments.

All route-study records report unavailable MQTT and Timescale endpoints; their samples used the local SQLite fallback, with online DAQ collection disabled. The route results therefore assess invocation and application-record handling, not the normal online DAQ path. Fixture-backed routes used deterministic input; direct REST created no task and did not invoke an external model.

A separate v0.8.3 onboarding replay used a fresh isolated application home and the documented C01 blueprint. The blueprint configured one Modbus/TCP device with four setpoints and seven DAQ nodes, plus MES REST scalar, 48-point vector, table, and image forms. Dedicated local MQTT and Timescale services were online for DAQ checks; object storage was unavailable, so image artifacts used the local-disk fallback. This was one configuration replay, not a route-study trial.

The approval-context suite changed each field of the approved line–product–recipe–run tuple for recipe apply, trial, and rollback with dispatch enabled (12 cases). A separate

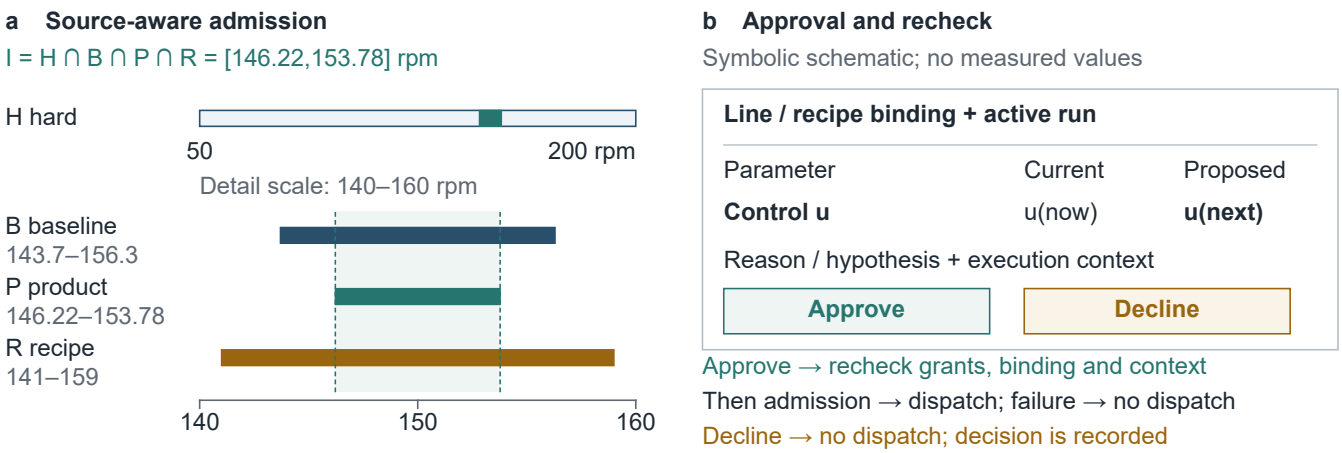


Fig. 4. Admission and approval boundaries. (a) Illustrative archived RPM configuration, not experimental data: online node interventions use the four-layer interval of Eq. (2), whereas recipe dispatch omits the previous recipe window. (b) Symbolic human approval binds a proposed action and its context tuple; execution rechecks the active line–product–recipe–run context, authority, and applicable limits. Approval does not reserve equipment state.

fault-injection case changed context after asynchronous grant revalidation but before the caller's final synchronous check. A test-only ablation removed only the tuple comparison while retaining the other checks. These outcomes were recorded in mock-driver state and are not process-level reliability estimates.

B. Registered and Direct-Call Route Results

All six model-mediated registered tasks completed and passed the task, approval, receipt, and sample-link checks (76 runner assertions). Cast-film settings of 125/130/125 rpm produced single 20-s samples of 45.69/46.69/45.69 μm. Annealing settings of 745 °C produced 679.60/677.70/678.10 °C. All six samples were below their target bands; they do not describe a response trajectory, steady state, or the quality of the model-selected settings.

The fixed-call harness replay linked both tasks to approval, receipt, and active-run samples (31 assertions); task completion was not asserted. Its cast-film and annealing samples were 46.37 μm and 674.80 °C, both below target. Direct REST returned verified receipts/readbacks and run-tagged samples in both scenes (19 checks); its samples were 46.37 μm and 675.20 °C, also below target. The fixture-backed command-line adapter completed both tasks and linked their samples (37 application and 28 isolation checks); its samples were 46.38 μm and 674.40 °C, also below target. The tested routes reached the shared operation services under two preconfigured signal and protocol mappings. They do not establish process attainment, configuration reuse, general adapter interchangeability, or portability to an unseen scene.

C. Blueprint-Driven Scenario Onboarding

On the v0.8.3 build, the C01 connectivity preflight passed 3/3 checks; line provisioning passed 20/20; channel construction passed 13/13; and all four MES forms returned records with the expected format and field dictionary. The V1–V6 verifier passed 23/23 checks, including sample availability for all seven DAQ nodes, range rejection, legal write/readback, recipe mapping, channel/plugin checks, and zero writes to a protected demonstration line. One smoke proposal used the test-only automatic-approval policy to change line speed from 37 to 39 m/min within the 28–48 m/min recipe window; the PLC twin confirmed the write. The approval was synthetic. These checks establish one documented blueprint-to-simulator onboarding instance, not human-review performance, film quality, process attainment, arbitrary protocol coverage, configuration reuse, repeatability, or physical transfer. Supplementary File S1 includes the inputs, health and DAQ status, runner output, and complete V1–V6 log.

TABLE II
ROUTE AND APPROVAL-CONTEXT CHECKS. ASSERTION COUNTS ARE SOFTWARE CHECKS, NOT INDEPENDENT PROCESS REPLICATIONS.

Execution path	Result and inference limit
Registered task harness	Six sequential tasks; 6/6 task-to-sample links and verified receipts; 76 runner assertions. One sample per task; all below target.
Fixed-call harness replay	One task per scene; 2/2 approval/receipt/sample links; 31 assertions. Task completion was not asserted.
Direct REST operation	One call per scene; 2/2 verified receipts/readbacks and run-tagged samples; 19 checks. No task edge.
Fixture-backed command-line task	One completed task per scene; 2/2 task-to-sample links; 37 application and 28 isolation checks. No external model decision.
Approval-context guard	12/12 injected tuple changes blocked before mock writes; queued between-check mutation refused before dispatch. Mock state only.

D. Approval-Context Guard and Ablation

The production handler blocked all 12 one-field mutations before a mock write or execution record. The queued context change was also refused before `recipe_apply` dispatch. In the ablation, removing the tuple comparison allowed six product/run changes through; each produced two mock writes and an execution record, while the other checks still blocked line/recipe changes. The local unit suite passed 566/566 checks across seven suites.

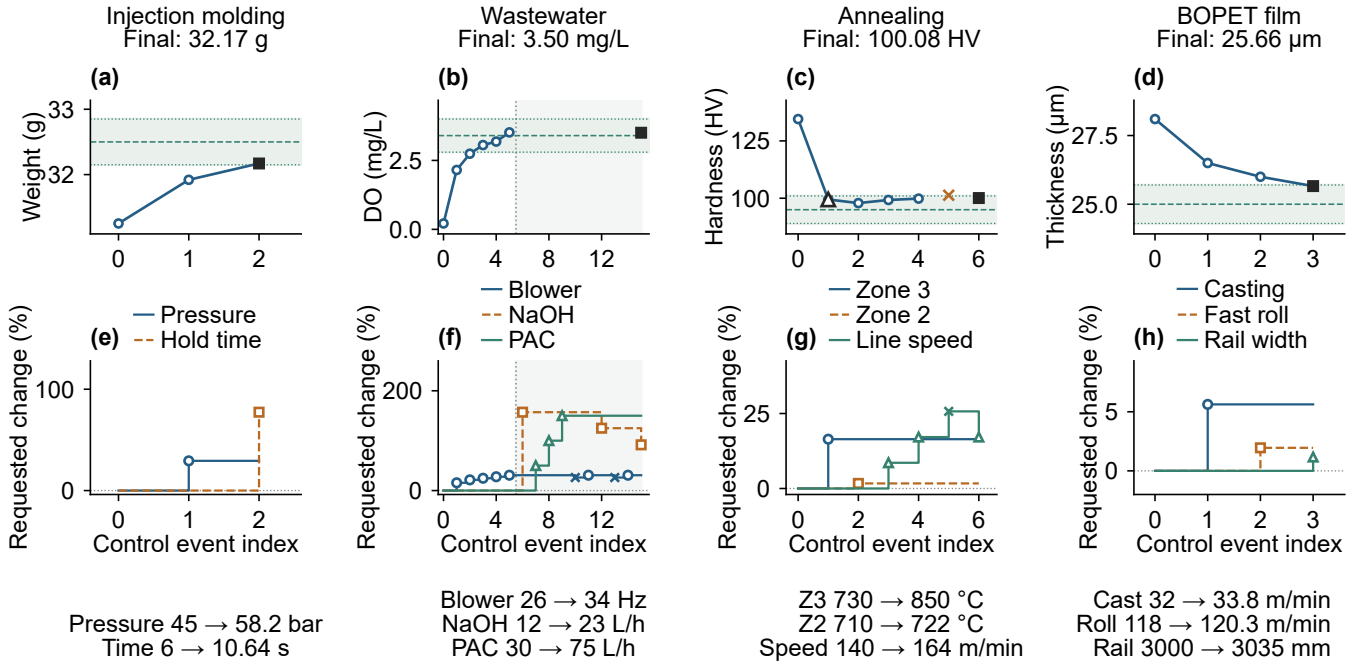


Fig. 5. Archived control sequences and outcomes for four simulator scenarios (seed 42; one scripted trajectory per scenario). Upper panels show PV observations and scenario-specific target windows; lower panels show requested setpoint changes from baseline. Black squares mark mission-final readings; for injection molding and BOPET they overlap the last observation. In wastewater treatment, later NaOH and PAC requests belong to pH and total-phosphorus stages, and the terminal DO reading is not joined to the DO trajectory. Crosses mark reverted requests. In annealing, the open triangle is the 99.46-HV event without a linked optimization record; the cross is the 101.3-HV event-text value after a reverted candidate, not a structured trajectory observation. Requests are not device readbacks. These historical direct-node missions are separate from the route study and do not show current route performance or a common optimizer score.

The ablation isolates the tuple comparison within the tested software state. It does not demonstrate atomicity during network or device I/O, safe physical operation, operator effectiveness, or end-to-end DAQ integrity.

E. Archived Four-Scenario Control Sequences and Outcomes

The current route study collected one post-trial process sample per task. Fig. 5 instead shows four archived seed-42 simulator missions from an earlier scripted direct-node interface. Each column pairs requested setpoint changes with process-variable (PV) observations and its own target window; event indices give order, not elapsed time.

In injection molding, hold pressure and time increased from 45 to 58.2 bar and 6 to 10.64 s; measured weight rose from 31.25 to 32.17 g, ending 0.02 g inside the 32.5 ± 0.35 g window. Five wastewater blower increments from 26 to 34 Hz raised dissolved oxygen (DO) from 0.214 to 3.517 mg/L, inside the 3.4 ± 0.6 mg/L window; later NaOH and PAC actions addressed separate pH and phosphorus stages. In annealing, zone-3 temperature requests rose from 730 to 850°C; later zone-2 and speed changes produced 97.88–99.84 HV, and the terminal value was 100.08 HV in the 95 ± 6 HV window. The 101.3 HV value after a reverted 176 m/min candidate appears only in event text. For BOPET, casting speed, fast-roll speed, and rail width changed from 32 to 33.8 m/min, 118 to 120.3 m/min, and 3000 to 3035 mm; thickness fell from 28.10 to 25.66 μm, ending 0.04 μm inside the upper edge of its 25.0 ± 0.7 μm window. There is one historical trajectory per scenario; the records illustrate scenario-specific feedback but not repeatability or current-route attainment.

F. Evidence Synthesis

The route study supports record continuity for the tested task and direct-call contracts across two preconfigured mappings; all twelve route-linked samples were below target. The separate onboarding replay passed the documented C01 configuration checks with isolated online MQTT/Timescale and one simulator- confirmed write. The mock-state comparison shows that the tested context check blocked stale requests before software dispatch. Fig. 5 is historical evidence and is not pooled with these results.

Together, these results distinguish command completion from process outcome: a verified readback confirms the setpoint path, whereas the later run-tagged sample indicates whether the process variable entered its target band. The study establishes selected integration behavior, one simulated onboarding instance, and a bounded software guard; it does not establish process improvement, general portability, or physical-plant performance. Supplementary File S1 includes the route inputs and logs, the separate onboarding replay, historical scenario records, and the evidence/hash manifests. The model-mediated tasks are not deterministic replays, and the route samples are not independent replicates.

VI. Discussion

A. Interpretation

AgentWorkShop composes collaborative-task services, production context, named operations, semantic signal access, configured device drivers, and service-owned records into a shared task-to-device path. The current experiments trace this design through task or direct-operation entry, approval-bound dispatch, readback receipt, and a later run-tagged process sample. Registered task routes preserve the task, approval, receipt, and sample associations; direct REST preserves operation, readback, and sample records without a task edge. The two simulator scenes use different signal and protocol mappings, so the observed joins support the tested composition and route-specific record contract. Separately, the v0.8.3 replay configured the documented C01 coating-line blueprint through the existing onboarding workflow, verified its MES forms and DAQ nodes, and confirmed one simulated write. This shows that one documented configuration can pass through that workflow; it is not a repeatability or general-portability result.

The process results bound the interpretation of that evidence. Cast-film samples range from 45.69 to 46.69 μm, or 1.31–2.31 μm below the 48 μm lower bound. Annealing samples range from 674.40 to 679.60 °C, or 20.40–25.60 °C below the 700 °C lower bound. These measurements are single snapshots taken 20 s after an operation, so they cannot establish a response curve, convergence, or the cause of the observed shortfall.

Fig. 5 pairs requested setpoint changes with the resulting process observations in four separate archived seed-42 simulator missions. The trajectories show scenario-specific responses, including the wastewater mission's later pH and total-phosphorus actions and the annealing candidate that was reverted. Their terminal observations lie within their own target windows, but each panel contains one historical scripted direct-node trajectory. The figure therefore explains the structure of those archived action–response records; it does not establish target attainment by current v0.8.0 routes or repeatability across runs.

This gap between command evidence and process evidence is central to the framework's execution model. A verified readback confirms that the requested setpoint reached the configured device; it does not show that the process variable entered its target band. A task can therefore complete its approved operation path while its associated process sample remains out of band. Keeping the receipt and run-tagged sample as separate, linked records lets the system report both facts without treating command completion as product attainment. The current results validate this distinction in the tested software paths, not an improvement in control quality.

The route evidence also depends on each entry contract. Registered model-mediated and fixture-backed CLI tasks retain task, approval, receipt, and sample links. The fixed-call replay exercises the registered harness with deterministic input but does not assert task completion. Direct REST reaches the same operation boundary and retains a verified receipt and run-tagged sample without a task edge. The fixture replay, direct REST, and CLI checks ran in separate isolated environments; the model-mediated triplets shared a persistent simulator per scene. These path-specific contexts and unequal sample counts mean their numerical similarity is not evidence of route equivalence. All route-study runs used the SQLite fallback because MQTT and Timescale were unavailable; the separate C01 onboarding replay exercised online MQTT/Timescale on isolated services. The results support the specified record contracts under two configured signal and protocol mappings and feasibility of one blueprint-driven simulated onboarding workflow. They do not establish interchangeable adapters, configuration reuse, or general portability to an unseen scene.

The context-guard test identifies the software check's contribution under mock state: comparing the approved line–product–recipe–run tuple blocked product/run mutations that the retained checks alone did not reject. This supports a bounded claim about software dispatch. It does not show atomicity across network or device I/O, safe physical operation, or operator effectiveness. Synthetic approval also does not measure human review performance [25]–[27].

B. Scope and Limitations

Only the reported adapter and scene configurations were exercised. The single C01 replay does not test repeatable onboarding, profile or binding reuse across lines, arbitrary unseen-scene configuration, or physical transfer. The evaluation also does not establish general adapter interchangeability, normal online DAQ operation in the route trials, or a platform baseline. The feature-level related-work comparison positions the implementation question; it does not show that AgentWorkShop is more efficient or more capable than another platform. A comparative study would need a shared task set, explicit integration-effort or record-completeness measures, and matched deployments.

The route study used one simulator instance per scene, three sequential model-mediated trials per scene, and one post-trial process sample per trial. These samples are not independent replicates and cannot establish a process response curve. Follow-up evaluation should vary configuration profiles under a controlled onboarding protocol, use repeated process trials, and compare against a defined alternative integration path. Operation restrictions do not establish security [28]–[30] or replace independent PLC/SIS safeguards [31].

Scope and channel-policy refusals are not uniformly persisted; auditability is limited to the records and denial classes described in Section IV.

VII. Conclusion

AgentWorkShop brings task adapters, production context, named operations, semantic I/O, configured protocol drivers, and service-owned records into a modular task-to-device framework. Tested task routes and direct calls used a shared operation boundary across cast-film and annealing simulator mappings while preserving route-specific records. A separate v0.8.3 replay completed one documented coating-line onboarding configuration with online DAQ checks and a

simulator-confirmed smoke write. Route-study samples remained below target, and a mock-state ablation isolated the effect of an approval-bound context check. The study establishes selected integration, one simulated onboarding instance, and software-contract behavior; it does not establish process improvement, configuration reuse, physical transfer, general portability, or comparative platform benefit.

REFERENCES

[1] Y. Xia, N. Jazdi, J. Zhang, C. Shah, and M. Weyrich, "Control industrial automation system with large language model agents," in *Proc. IEEE 30th Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205539.

[2] K.-C. Huang, H.-T. Lin, C.-H. Lin, and M.-T. Huang, "Large language model integration in industrial cyber-physical systems: A multi-modal architecture for automated control generation," in *Proc. Int. Conf. on Control, Automation and Diagnosis (ICCAD)*, 2026, pp. 1–6, doi: 10.1109/ICCAD69956.2026.11643124.

[3] K. Köhle, D. Anicic, T. A. Runkler, and R. Graf, "LLM-driven multi-agent control for skill-based smart manufacturing," in *Proc. IEEE 31st Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*, 2026, pp. 1–8, doi: 10.1109/ETFA68377.2026.11725257.

[4] M. Waseem, K. Bhatta, C. Li, and Q. Chang, "Pretrained LLMs as real-time controllers for robot operated serial production line," *Mach. Learn.: Eng.*, vol. 2, no. 1, 2026, art. no. 015006; doi: 10.1088/3049-4761/ae4ee8.

[5] Z. Liu *et al.*, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026, doi: 10.1109/TSE.2026.3667895.

[6] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179, doi: 10.1109/CDC57313.2025.11312156.

[7] J. Lim and I. Kovalenko, "A large language model-enabled control architecture for dynamic resource capability exploration in multi-agent manufacturing systems," in *Proc. IEEE 21st Int. Conf. on Automation Science and Engineering (CASE)*, 2025, pp. 2088–2095, doi: 10.1109/CASE58245.2025.11163802.

[8] Z. Zhao *et al.*, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888; doi: 10.1016/j.aei.2025.103888.

[9] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205597.

[10] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025. [Online]. Available: https://arxiv.org/abs/2510.11300v2

[11] C. Graul, W. Żarski, D. Mikołajewski, and I. Rojek, "Ai agents in industry 4.0: AAS–OPC UA–LLM architecture as the foundation of intelligent manufacturing systems in the context of industrial enterprise implementation," *Applied Sciences*, vol. 16, no. 11, p. 5428, 2026, doi: 10.3390/app16115428.

[12] D. Schall, "Safe integration of large language models into industrial process control: A multi-agent architecture with P&ID-grounded validation," *Autonomous Intelligent Systems*, vol. 6, 2026, article no. 14; doi: 10.1007/s43684-026-00136-1.

[13] S.-C. Lin and C.-Y. Chen, "An integrated framework featuring policy-governed agentic AI for closed-loop manufacturing control with multi-source sensor–MES–ERP," *The International Journal of Advanced Manufacturing Technology*, vol. 143, pp. 6405–6428, 2026, doi: 10.1007/s00170-026-17806-2.

[14] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber–physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025, doi: 10.1109/TII.2024.3514178.

[15] Y. Xia, N. Jazdi, and M. Weyrich, "An architecture for integrating large language models with digital twins and automation systems," in *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–8, doi: 10.1109/ETFA65518.2025.11205636.

[16] L. M. Vieira da Silva, A. Köcher, and F. Gehlhoff, "Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing," in *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2025, pp. 1–4, doi: 10.1109/ETFA65518.2025.11205601.

[17] K. Köhle, D. Anicic, T. A. Runkler, K. Dorofeev, A. S. Thuluva, and R. Graf, "Virtually commissioned semantic recipes based on OPC UA skills," in *2025 IEEE 8th International Conference on Industrial Cyber-Physical Systems (ICPS)*, 2025, pp. 1–6, doi: 10.1109/ICPS65515.2025.11087832.

[18] X. Jiang *et al.*, "Agentic AI for safety-aware process monitoring and fault diagnosis: A review," *Processes*, vol. 14, no. 13, 2026, art. no. 2112; doi: 10.3390/pr14132112.

[19] Model Context Protocol contributors, "Model Context Protocol: Tools," Specification, version 2026-07-28, 2026, accessed: Oct. 20, 2026. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[20] ISA, "ISA-88.00.01-2010: Batch control—part 1: Models and terminology," International Standard, 2010.

[21] ——, "ANSI/ISA-95.00.01-2025 (IEC 62264-1 mod): Enterprise-control system integration—part 1: Models and terminology," International Standard, 2025.

[22] J. Moyne, E. del Castillo, and A. M. Hurwitz, Eds., *Run-to-Run Control in Semiconductor Manufacturing*, 1st ed. Boca Raton, FL: CRC Press, 2000.

[23] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. on Software Engineering (ICSE)*, 2026, pp. 2938–2950, doi: 10.1145/3744916.3764546.

[24] E. Debenedetti *et al.*, "Defeating prompt injections by design," 2025, arXiv preprint arXiv:2503.18813. [Online]. Available: https://arxiv.org/abs/2503.18813

[25] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.

[26] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000, doi: 10.1109/3468.844354.

[27] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983, doi: 10.1016/0005-1098(83)90046-8.

[28] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[29] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1831–1847. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[30] IEC, "IEC 62443-3-3:2013: Industrial communication networks—network and system security—part 3-3: System security requirements and security levels," Standard, 2013.

[31] ——, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.